02-algorithms-data-structures

02. Algorithms & Data Structures

Level: Beginner
Jam: 8 (4 minggu × 2 sesi)
Prasyarat: JavaScript Fundamentals
Output: Paham Big O, struktur data dasar, bisa ngerjain LeetCode easy/medium

Kenapa Ini Penting?

`Tanpa Algo & DS: Bisa bikin web app, tapi lemot & ga scalable.`
`Pake Algo & DS: Bikin app yang efficient, lolos tech interview.`

> "Bad programmers worry about the code. Good programmers worry about data structures and their relationships."
> — Linus Torvalds

Materi

Sesi	Topik	File
1	Big O Notation — ngukur kecepatan kode	01-big-o.md
2	Arrays & Hash Tables — struktur data paling dipake	02-arrays-hashtables.md
3	Stacks, Queues, Linked Lists	03-stacks-queues-linkedlists.md
4	Recursion & Sorting	04-recursion-sorting.md
5	Trees & Graphs (pengantar)	05-trees-graphs.md
6	LeetCode Practice — dari brute force ke optimal	06-leetcode-practice.md

Tools

- TypeScript (sama kayak modul sebelumnya)
- LeetCode / HackerRank / AtCoder — akun gratis
- Big O Cheat Sheet: bigocheatsheet.com

2.1 Big O Notation

Big O = cara ngukur seberapa cepet kode jalan, dalam jumlah langkah (bukan detik).

Analogi

> "Cari buku 'JavaScript' di rak 1000 buku."

Cara	Langkah	Big O
Baca satu-satu dari awal	1000 langkah	O(n)
Mulai dari tengah (rak berurutan abjad)	~10 langkah	O(log n)
Tau persis posisinya	1 langkah	O(1)

Jenis Big O (dari paling cepet)

```
O(1)      → Constant   — Ga peduli seberapa besar input
O(log n)  → Logarithmic — Makin besar input, tambahannya dikit
O(n)      → Linear     — Proporsional sama input
O(n log n)→ Linearithmic — Sorting efficient
O(n²)     → Quadratic  — Loop di dalem loop (LAMBAT!)
O(2^n)    → Exponential — Jangan dipake (fibonacci rekursif)
```

Contoh Kode

```ts
// O(1) — Constant: langsung akses index
function getFirst(arr: number[]): number {
  return arr[0]; // 1 langkah, berapapun size arr
}

// O(n) — Linear: iterasi semua item
function findMax(arr: number[]): number {
  let max = -Infinity;
  for (const num of arr) {  // n langkah
    if (num > max) max = num;
  }
  return max;
}

// O(n²) — Quadratic: nested loop
function findDuplicates(arr: number[]): boolean {
  for (let i = 0; i < arr.length; i++) {       // n
    for (let j = i + 1; j < arr.length; j++) {  // n
```

```
      if (arr[i] === arr[j]) return true;
    }
  }
  return false;
}
// Input 10 item → 100 langkah. Input 1000 → 1.000.000 langkah. NGERI!
```

Aturan Sederhana

1. **Drop constants:** O(2n) = O(n). O(100) = O(1)
2. **Drop non-dominants:** O(n + n^2) = O(n^2). Ambil yang paling gede
3. **Different inputs → different variables:** Input a dan b → O(a + b), bukan O(2n)

Space Complexity

```
// O(1) — ga pake memory tambahan
function sum(arr: number[]): number {
  let total = 0;  // cuma 1 variable
  for (const n of arr) total += n;
  return total;
}

// O(n) — bikin array baru sebesar input
function double(arr: number[]): number[] {
  return arr.map(n => n * 2);  // array baru sepanjang arr
}
```

Latihan Big O

Tentukan Big O dari:

```
// Soal 1
function mystery(arr: number[]) {
  console.log(arr[0]);
  console.log(arr[Math.floor(arr.length / 2)]);
  console.log(arr[arr.length - 1]);
}

// Soal 2
function containsPair(arr: number[], target: number): boolean {
  const seen = new Set<number>();
  for (const num of arr) {
    if (seen.has(target - num)) return true;
```

```
    seen.add(num);
  }
  return false;
}

// Soal 3
function printPairs(arr: number[]) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = 0; j < arr.length; j++) {
      console.log(arr[i], arr[j]);
    }
  }
}
```

Jawaban: Soal 1 = O(1), Soal 2 = O(n), Soal 3 = O(n²)

2.2 Arrays & Hash Tables

Dua struktur data paling sering dipake. 90% problem LeetCode pake ini.

Array

```
const arr: number[] = [1, 2, 3, 4, 5];
```

Operasi	Big O	Catatan
Access (arr[i])	O(1)	Langsung ke index
Search (cari nilai)	O(n)	Kalo ga tau indexnya
Push (tambah di akhir)	O(1)	Umumnya
Insert/Delete di awal	O(n)	Geser semua index!
Insert/Delete di akhir	O(1)	Kalo array dynamic

Hash Table (Map / Object / Set)

```
// Object
const user: Record<string, any> = { nama: "Budi", umur: 17 };

// Map (lebih proper)
const scores = new Map<string, number>([
  ["Budi", 85],
  ["Andi", 90],
]);
```

```ts
// Set (unique values)
const unique = new Set([1, 2, 2, 3, 3, 3]);  // {1, 2, 3}
```

Operasi	Big O	Catatan
Access by key	O(1)	Langsung ke key
Insert	O(1)	
Delete	O(1)	
Search key	O(1)	Cepet banget

Array vs Hash Table — Kapan Pake yang Mana?

Array	Hash Table
Butuh urutan	Ga peduli urutan
Butuh akses index	Butuh akses KEY (nama/ID)
Iterasi semua item	Cek keberadaan item
Memory efficient	Lebih boros memory

Pattern Penting

Two Pointers (Array)

```ts
function isPalindrome(str: string): boolean {
  let left = 0;
  let right = str.length - 1;

  while (left < right) {
    if (str[left] !== str[right]) return false;
    left++;
    right--;
  }
  return true;
}
// Big O: O(n), Space: O(1)
```

Frequency Counter (Hash Table)

```ts
function areAnagrams(s1: string, s2: string): boolean {
  if (s1.length !== s2.length) return false;

  const freq = new Map<string, number>();

  for (const char of s1) {
```

```
    freq.set(char, (freq.get(char) || 0) + 1);
  }

  for (const char of s2) {
    const count = freq.get(char);
    if (!count) return false;
    freq.set(char, count - 1);
  }

  return true;
}
// Big O: O(n), Space: O(n)
```

Sliding Window (Array)

```
function maxSubarraySum(arr: number[], k: number): number {
  let maxSum = 0;
  let windowSum = 0;

  // Inisialisasi window pertama
  for (let i = 0; i < k; i++) {
    windowSum += arr[i];
  }
  maxSum = windowSum;

  // Geser window
  for (let i = k; i < arr.length; i++) {
    windowSum = windowSum - arr[i - k] + arr[i];
    maxSum = Math.max(maxSum, windowSum);
  }

  return maxSum;
}
// Big O: O(n), Space: O(1)
```

LeetCode Problems

Problem	Pattern	Difficulty
Two Sum	Hash Map	Easy
Contains Duplicate	Set	Easy
Valid Anagram	Frequency Counter	Easy
Group Anagrams	Hash Map	Medium
Product of Array Except Self	Prefix/Suffix	Medium

Latihan

1. Implement `containsDuplicate` pake Set (larangan nested loop)
2. Implement `twoSum` pake Map (O(n), bukan O(n²))
3. Implement `maxSubarraySum` sliding window
4. Cek anagram pake frequency counter

2.3 Stacks, Queues & Linked Lists

Stack (Tumpukan)

Prinsip: LIFO — Last In, First Out

Analogi: tumpukan piring. Yang paling atas (terakhir ditambah) adalah yang pertama diambil.

```
class Stack<T> {
  private items: T[] = [];

  push(item: T): void { this.items.push(item); }
  pop(): T | undefined { return this.items.pop(); }
  peek(): T | undefined { return this.items[this.items.length - 1]; }
  isEmpty(): boolean { return this.items.length === 0; }
  size(): number { return this.items.length; }
}

// Contoh: Browser History
const history = new Stack<string>();
history.push("google.com");
history.push("github.com");
history.push("mastra.ai");

console.log(history.pop());  // mastra.ai (back button)
console.log(history.pop());  // github.com
```

Operasi	Big O
Push	O(1)
Pop	O(1)
Peek	O(1)

LeetCode: Valid Parentheses

```
function isValid(s: string): boolean {
  const stack: string[] = [];
  const pairs: Record<string, string> = {
```

```
    ')': '(',
    ']': '[',
    '}': '{',
  };

  for (const char of s) {
    if (char === '(' || char === '[' || char === '{') {
      stack.push(char);
    } else {
      if (stack.pop() !== pairs[char]) return false;
    }
  }

  return stack.length === 0;
}
// O(n)
```

Queue (Antrian)

Prinsip: FIFO — First In, First Out

Analogi: antrian kasir. Yang pertama dateng, pertama dilayani.

```
class Queue<T> {
  private items: T[] = [];

  enqueue(item: T): void { this.items.push(item); }
  dequeue(): T | undefined { return this.items.shift(); }  // O(n)!
  front(): T | undefined { return this.items[0]; }
  isEmpty(): boolean { return this.items.length === 0; }
}

// Contoh: Print Queue
const printQueue = new Queue<string>();
printQueue.enqueue("Dokumen 1");
printQueue.enqueue("Dokumen 2");
printQueue.enqueue("Dokumen 3");

console.log(printQueue.dequeue());  // "Dokumen 1" (yang pertama dateng, pertama
console.log(printQueue.dequeue());  // "Dokumen 2"
```

> **Catatan:** `shift()` di array JS itu O(n) karena geser element. Untuk queue yang O(1), pake linked list atau new `Queue()` dari library.

Linked List

Node = data + pointer ke node berikutnya

```
[1 | *] -> [2 | *] -> [3 | null]
```

```
class ListNode<T> {
  constructor(
    public value: T,
    public next: ListNode<T> | null = null
  ) {}
}

class LinkedList<T> {
  head: ListNode<T> | null = null;

  append(value: T): void {
    const node = new ListNode(value);
    if (!this.head) {
      this.head = node;
      return;
    }
    let current = this.head;
    while (current.next) {
      current = current.next;
    }
    current.next = node;
  }

  prepend(value: T): void {
    const node = new ListNode(value);
    node.next = this.head;
    this.head = node;
  }

  print(): void {
    let current = this.head;
    const values: T[] = [];
    while (current) {
      values.push(current.value);
      current = current.next;
    }
    console.log(values.join(' -> '));
  }
}

const list = new LinkedList<number>();
```

```
list.append(10);
list.append(20);
list.prepend(5);
list.print();  // 5 -> 10 -> 20
```

Operasi	Array	Linked List
Access by index	O(1) □	O(n) □
Insert/Delete di awal	O(n) □	O(1) □
Insert/Delete di akhir	O(1) □	O(n) □
Search	O(n) □	O(n) □

LeetCode: Reverse Linked List

Latihan

1. Stack: implement undo/redo system sederhana
2. Queue: implement task scheduler (proses satu-satu)
3. Linked List: implement reverse (pusing? wajar, ini soal tersulit di sesi ini)
4. Valid Parentheses — solve pake stack tanpa liat jawaban

2.4 Recursion & Sorting

Recursion

Function yang manggil dirinya sendiri.

2 komponen WAJIB:

1. **Base case** — kapan berhenti
2. **Recursive case** — manggil diri sendiri dengan input lebih kecil

```
// Factorial: n! = n * (n-1) * (n-2) * ... * 1
function factorial(n: number): number {
  // Base case
  if (n <= 1) return 1;
  // Recursive case
  return n * factorial(n - 1);
}

console.log(factorial(5)); // 120

// Call stack:
// factorial(5) = 5 * factorial(4)
```

```
//   factorial(4) = 4 * factorial(3)
//     factorial(3) = 3 * factorial(2)
//       factorial(2) = 2 * factorial(1)
//         factorial(1) = 1 ← base case!
//       = 2 * 1 = 2
//     = 3 * 2 = 6
//   = 4 * 6 = 24
// = 5 * 24 = 120

// Fibonacci (NAIVE — jangan pake ini, O(2^n)!)
function fib(n: number): number {
  if (n <= 1) return n;
  return fib(n - 1) + fib(n - 2);
}

// Fibonacci (OPTIMAL — memoization)
function fibMemo(n: number, memo: Record<number, number> = {}): number {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
  return memo[n];
}
// O(n) — simpan hasil yang udah dihitung
```

Sorting

Bubble Sort (O(n²)) — Jangan dipake di production

```
function bubbleSort(arr: number[]): number[] {
  const result = [...arr];
  for (let i = 0; i < result.length; i++) {
    for (let j = 0; j < result.length - 1 - i; j++) {
      if (result[j] > result[j + 1]) {
        [result[j], result[j + 1]] = [result[j + 1], result[j]];
      }
    }
  }
  return result;
}
// Pahamin konsepnya aja, jangan pake beneran. .sort() built-in udah O(n log n)
```

Merge Sort (O(n log n)) — Divide & Conquer

```
function mergeSort(arr: number[]): number[] {
  if (arr.length <= 1) return arr;
```

```
  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));

  return merge(left, right);
}

function merge(left: number[], right: number[]): number[] {
  const result: number[] = [];
  let i = 0, j = 0;

  while (i < left.length && j < right.length) {
    if (left[i] < right[j]) {
      result.push(left[i]);
      i++;
    } else {
      result.push(right[j]);
      j++;
    }
  }

  return [...result, ...left.slice(i), ...right.slice(j)];
}

console.log(mergeSort([38, 27, 43, 3, 9, 82, 10]));
// [3, 9, 10, 27, 38, 43, 82]
```

Visual: Merge Sort Animation

Quick Sort (O(n log n) average)

```
function quickSort(arr: number[]): number[] {
  if (arr.length <= 1) return arr;

  const pivot = arr[arr.length - 1];
  const left: number[] = [];
  const right: number[] = [];

  for (let i = 0; i < arr.length - 1; i++) {
    if (arr[i] < pivot) {
      left.push(arr[i]);
    } else {
      right.push(arr[i]);
    }
  }
```

```
  return [...quickSort(left), pivot, ...quickSort(right)];
}
```

Latihan

1. Implement `factorial` pake recursion
2. Implement `power(base, exp)` — hitung pangkat pake recursion
3. Implement `fibonacci` pake memoization
4. Implement sorting array pake `.sort()` — kapan harus pake custom sort?

2.5 Trees & Graphs (Pengantar)

Tree (Pohon)

Struktur data hierarkis: root → child → grandchild

```
      1  ← root
     / \
    2   3
   / \   \
  4   5   6  ← leaves (daun)
```

Binary Tree

```
class TreeNode<T> {
  constructor(
    public value: T,
    public left: TreeNode<T> | null = null,
    public right: TreeNode<T> | null = null
  ) {}
}

// Bikin tree:
//      10
//     /  \
//    5    15
//   / \
// 3    7
const root = new TreeNode(10);
root.left = new TreeNode(5);
root.right = new TreeNode(15);
root.left.left = new TreeNode(3);
root.left.right = new TreeNode(7);
```

Binary Search Tree (BST)

Aturan: Left child < parent < right child

```ts
function searchBST(root: TreeNode<number> | null, target: number): boolean {
  if (!root) return false;

  if (target === root.value) return true;
  if (target < root.value) return searchBST(root.left, target);
  return searchBST(root.right, target);
}
// O(log n) — setiap langkah buang setengah tree!
```

Tree Traversal

```ts
// DFS - In-order (kiri, root, kanan) → hasilnya sorted
function inOrder(node: TreeNode<number> | null): void {
  if (!node) return;
  inOrder(node.left);
  console.log(node.value);
  inOrder(node.right);
}

// BFS - Level order (pakai queue)
function bfs(root: TreeNode<number> | null): void {
  const queue: TreeNode<number>[] = [];
  if (root) queue.push(root);

  while (queue.length > 0) {
    const node = queue.shift()!;  // dequeue
    console.log(node.value);
    if (node.left) queue.push(node.left);
    if (node.right) queue.push(node.right);
  }
}
```

Graph (Graf)

Node + connections (edges). Bisa satu arah (directed) atau dua arah (undirected).

```ts
// Adjacency List — representasi graph paling umum
type Graph = Map<string, string[]>;

const graph: Graph = new Map([
  ['A', ['B', 'C']],
```

```
  ['B', ['A', 'D', 'E']],
  ['C', ['A', 'F']],
  ['D', ['B']],
  ['E', ['B', 'F']],
  ['F', ['C', 'E']],
]);

// BFS traversal
function bfsGraph(graph: Graph, start: string): void {
  const visited = new Set<string>();
  const queue: string[] = [start];

  while (queue.length > 0) {
    const node = queue.shift()!;
    if (visited.has(node)) continue;

    visited.add(node);
    console.log(node);

    for (const neighbor of graph.get(node) || []) {
      if (!visited.has(neighbor)) {
        queue.push(neighbor);
      }
    }
  }
}
```

Kapan Dipake?

Struktur	Use Case
BST	Database index, dictionary
Heap	Priority queue (scheduling)
Trie	Autocomplete, spell checker
Graph	Social network, maps, routing

LeetCode

Problem	Difficulty
Max Depth of Binary Tree	Easy
Invert Binary Tree	Easy
Same Tree	Easy

Latihan

1. Implement search di BST (iterative, bukan recursive)
2. Implement maxDepth — dalem mana tree-nya?
3. Invert binary tree — pernah viral siapa sanggup?

2.6 LeetCode Practice — Dari Brute ke Optimal

Cara Ngerjain LeetCode

Step-by-step approach:

1. **Pahamin problem** — contoh input/output, edge cases
2. **Brute force dulu** — yang penting jalan, ga mikir optimal
3. **Optimize** — dari O(n²) → O(n log n) → O(n)
4. **Write code** — pake TypeScript, run di LeetCode
5. **Test** — edge cases: empty, besar, negatif, duplicated

> "First, make it work. Then, make it fast."

10 Problem Progression

Level 1: Easy (Foundation)

1. Two Sum

```typescript
// Brute: O(n²) — nested loop
function twoSumBrute(nums: number[], target: number): number[] {
  for (let i = 0; i < nums.length; i++) {
    for (let j = i + 1; j < nums.length; j++) {
      if (nums[i] + nums[j] === target) return [i, j];
    }
  }
  return [];
}

// Optimal: O(n) — pake Map
function twoSum(nums: number[], target: number): number[] {
  const seen = new Map<number, number>();

  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];
    if (seen.has(complement)) {
      return [seen.get(complement)!, i];
    }
```

```
    seen.set(nums[i], i);
  }
  return [];
}
```

2. Valid Parentheses

```
function isValid(s: string): boolean {
  const stack: string[] = [];
  const pairs: Record<string, string> = {
    ')': '(', ']': '[', '}': '{',
  };

  for (const char of s) {
    if ('([{'.includes(char)) {
      stack.push(char);
    } else {
      if (stack.pop() !== pairs[char]) return false;
    }
  }

  return stack.length === 0;
}
```

3. Palindrome Number

```
function isPalindrome(x: number): boolean {
  if (x < 0) return false;
  if (x < 10) return true;
  if (x % 10 === 0) return false;  // 10, 20, 30...

  let reversed = 0;
  let original = x;

  while (original > reversed) {
    reversed = reversed * 10 + original % 10;
    original = Math.floor(original / 10);
  }

  return original === reversed || original === Math.floor(reversed / 10);
}
// O(log n) — ga perlu konversi ke string!
```

Level 2: Easy-Medium

4. Best Time to Buy and Sell Stock

```typescript
function maxProfit(prices: number[]): number {
  let minPrice = Infinity;
  let maxProfit = 0;

  for (const price of prices) {
    if (price < minPrice) {
      minPrice = price;  // beli di harga terendah
    } else if (price - minPrice > maxProfit) {
      maxProfit = price - minPrice;  // jual di harga tertinggi
    }
  }

  return maxProfit;
}
// O(n), Space: O(1)
```

5. Maximum Subarray (Kadane's Algorithm)

```typescript
function maxSubArray(nums: number[]): number {
  let maxCurrent = nums[0];
  let maxGlobal = nums[0];

  for (let i = 1; i < nums.length; i++) {
    maxCurrent = Math.max(nums[i], maxCurrent + nums[i]);
    maxGlobal = Math.max(maxGlobal, maxCurrent);
  }

  return maxGlobal;
}
// O(n) — ini sering ditanya di interview!
```

6. Contains Duplicate

```typescript
function containsDuplicate(nums: number[]): boolean {
  const seen = new Set<number>();

  for (const num of nums) {
    if (seen.has(num)) return true;
    seen.add(num);
  }

  return false;
}
// O(n) — jauh lebih cepet dari nested loop O(n²)
```

Level 3: Medium

7. Product of Array Except Self

```typescript
function productExceptSelf(nums: number[]): number[] {
  const result: number[] = new Array(nums.length).fill(1);

  // Prefix products
  let prefix = 1;
  for (let i = 0; i < nums.length; i++) {
    result[i] = prefix;
    prefix *= nums[i];
  }

  // Suffix products
  let suffix = 1;
  for (let i = nums.length - 1; i >= 0; i--) {
    result[i] *= suffix;
    suffix *= nums[i];
  }

  return result;
}
// O(n), Space: O(1) (result array ga diitung)
```

8. Longest Substring Without Repeating Characters

```typescript
function lengthOfLongestSubstring(s: string): number {
  const seen = new Map<string, number>();
  let maxLen = 0;
  let left = 0;

  for (let right = 0; right < s.length; right++) {
    const char = s[right];

    if (seen.has(char) && seen.get(char)! >= left) {
      left = seen.get(char)! + 1;
    }

    seen.set(char, right);
    maxLen = Math.max(maxLen, right - left + 1);
  }

  return maxLen;
}
// O(n) — sliding window + hash map
```

Level 4: Challenge

9. Three Sum

```typescript
function threeSum(nums: number[]): number[][] {
  nums.sort((a, b) => a - b);  // O(n log n)
  const result: number[][] = [];

  for (let i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] === nums[i - 1]) continue;  // skip duplicates

    let left = i + 1;
    let right = nums.length - 1;

    while (left < right) {
      const sum = nums[i] + nums[left] + nums[right];

      if (sum === 0) {
        result.push([nums[i], nums[left], nums[right]]);
        while (left < right && nums[left] === nums[left + 1]) left++;
        while (left < right && nums[right] === nums[right - 1]) right--;
        left++;
        right--;
      } else if (sum < 0) {
        left++;
      } else {
        right--;
      }
    }
  }

  return result;
}
// O(n²) — optimal untuk 3-sum
```

10. Merge Intervals

```typescript
function merge(intervals: number[][]): number[][] {
  if (intervals.length <= 1) return intervals;

  intervals.sort((a, b) => a[0] - b[0]);
  const result: number[][] = [intervals[0]];

  for (let i = 1; i < intervals.length; i++) {
    const last = result[result.length - 1];
    const current = intervals[i];
```

```
    if (current[0] <= last[1]) {
      last[1] = Math.max(last[1], current[1]);
    } else {
      result.push(current);
    }
  }

  return result;
}
// O(n log n) — sorting dulu, merge linear
```

Progress Tracker

#	Problem	Difficulty	Brute	Optimal	Selesai
1	Two Sum	Easy	$O(n^2)$	O(n)	☐
2	Valid Parentheses	Easy	—	O(n)	☐
3	Palindrome Number	Easy	O(n)	O(log n)	☐
4	Best Time to Buy & Sell	Easy	$O(n^2)$	O(n)	☐
5	Maximum Subarray	Easy	$O(n^2)$	O(n)	☐
6	Contains Duplicate	Easy	$O(n^2)$	O(n)	☐
7	Product Except Self	Medium	$O(n^2)$	O(n)	☐
8	Longest Substring	Medium	$O(n^3)$	O(n)	☐
9	Three Sum	Medium	$O(n^3)$	$O(n^2)$	☐
10	Merge Intervals	Medium	$O(n^2)$	O(n log n)	☐

Tips Interview

1. **Jangan langsung optimal** — interviewer mau liat proses mikir lo
2. **Voice your thought** — ”Ini $O(n^2)$ pake nested loop. Tapi bisa di-optimize pake hash map jadi O(n)”
3. **Handle edge cases** — empty, singular, negative, duplicate
4. **Test pake contoh** — ”Kalo inputnya [], outputnya harus []”

Next step: Bikin akun LeetCode, mulai dari Two Sum, naik perlahan. Target: 1 problem per hari = 30 problem/bulan.